System Analysis & Design

Project Title: Telecom Customer Retention Intelligence & Churn Risk Prediction

Submission Date: November 6, 2026

Dataset Reference: IBM Telco Customer Churn (7043 Records, 21 Logical Attributes)

1. Problem Statement & Objectives

1.1. Problem Statement & Goals

The primary objective of Churn Radar is to deliver an end-to-end data analysis and machine learning solution designed to predict customer churn for telecommunications providers. Operating on the IBM Telco Customer Churn dataset, the system processes 7,043 customer profile records containing demographic parameters, subscribed communication features, and financial billing contracts. By applying predictive classification modeling, Churn Radar identifies high-risk customers, highlights key churn drivers, and serves actionable retention insights through an interactive Streamlit web dashboard.

1.2. Core Functional Objectives

- **Data Standardization & Processing:** Clean, normalize, and encode raw tabular records to support both analytical database querying and machine learning model ingestion.
- **Predictive Churn Analytics:** Train and evaluate multiple supervised classification algorithms (Logistic Regression, Random Forest, and XGBoost) to accurately identify customers at risk of canceling service.
- **Interactive Business Intelligence:** Deploy an intuitive Streamlit interface allowing stakeholder interaction, individual churn probability scoring, and key performance metric visual dashboards.

1.3 System Requirements

Functional Requirements (FR):

- Ingest, clean, and encode 21 dataset attributes.
- Load pre-trained models (Logistic Regression, Random Forest, XGBoost) to generate real-time churn predictions.
- Host an interactive Streamlit UI for user input and visual output.

Non-Functional Requirements (NFR):

- Prediction response time under 1.5 seconds per query.
- Intuitive, user-friendly dashboard interface.
- Modular 3-Tier architecture (Frontend UI → Python Analytics Backend → SQL Database).

1.4 Software Architecture

Churn Radar uses a 3-Tier Client-Server Architecture:

- **Presentation Layer (Frontend):** Interactive UI built using Streamlit, capturing user inputs and presenting output metrics, risk scores, and visual charts.
- **Application Layer (Backend):** Python inference engine handling feature transformations, loading saved model binaries (.pkl/.joblib), and executing predictive classification scoring.
- **Data Layer (Storage):** Relational SQL Database storing normalized tables (customer, dependent, services, billing) for query execution and analytical reporting.

2. Database Design & Data Modeling

2.1 Entity-Relationship Diagram (ERD)

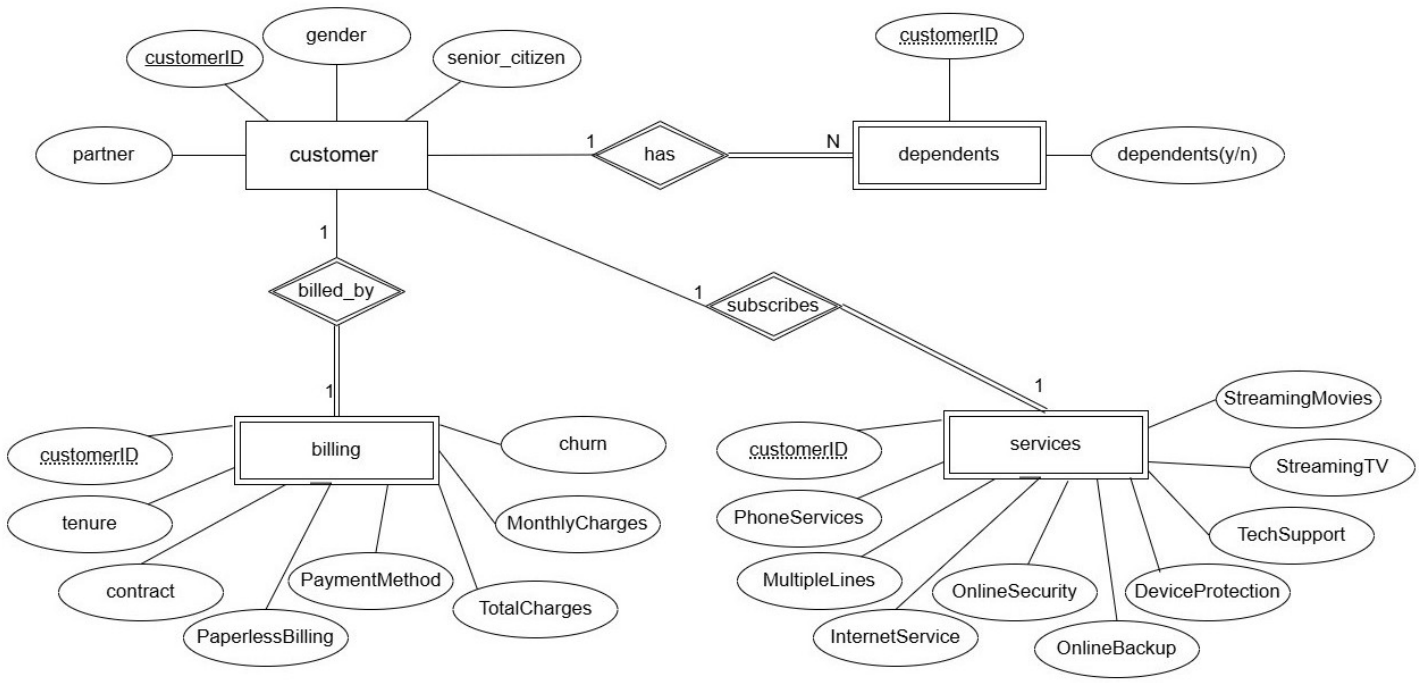


Figure 2.1: *Churn Radar Conceptual Entity-Relationship Diagram (ERD Notation)*

2.2 ERD Architectural Justification

This design splits account information into functional provisioning and billing systems while keeping the diagram clean and easy to read. Later in our machine learning pipeline, we join these tables back together using customerID to create a single clean dataset for training our models.

2.3 Logical & Physical Schema

- customer (Strong Entity):

  Primary Key: customerID

  Attributes: gender, senior_citizen, partner

  Description: Represents individual telecom subscribers and serves as the primary anchor entity for the entire system schema.

- dependents (Weak Entity):

  Foreign Key: customerID

  Attributes: dependents(y/n)

  Identifying Relationship: Connected to customer by the has relationship (1:N cardinality) with total participation.

- services (Weak Entity):

  Attributes (9 Service Features): PhoneServices, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies.

  Identifying Relationship: Connected to customer by the subscribes relationship (1:1 cardinality) with total participation.

- billing (Weak Entity):

  Attributes (Financial & Target Metrics): tenure, contract, PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges, churn.

  Identifying Relationship: Connected to customer by the billed_by relationship (1:1 cardinality) with total participation.

- Schema Normalization:
  1NF: Ensured by storing single atomic values in all attribute ovals without repeating groups.
  2NF: Ensured because non-key attributes fully depend on their parent entity's primary key through total participation identifying relationships.
  3NF: Ensured by eliminating transitive dependencies, ensuring functional dependencies route directly through customerID during relational mapping.

3. Data Flow & System Behavior

3.1 DFD (Data Flow Diagram)

- **Context Diagram (Level 0 DFD):**

  The Retention Analyst interacts with the Churn Radar System as an external entity. The user inputs customer profile variables, and the system returns predicted churn probability scores alongside interactive analytics.

- **Level 1 DFD:**

1. **Process 1.0 (Input & Preprocessing):** Accepts user inputs, cleans missing values, and normalizes numeric values.
2. **Process 2.0 (Model Execution):** Ingests scaled feature data into the saved machine learning pipeline (.pkl).
3. **Process 3.0 (Visualization & Scoring):** Outputs risk categorization (Low/Medium/High) to the Streamlit UI and updates BI metrics.
4. **Data Stores: D1: Customer Database, D2:** Trained Model Storage.

3.2 System Sequence Diagram

1. **Analyst → Streamlit UI:** User enters customer profile parameters into the web application interface.
2. **Streamlit UI → Python Engine:** Frontend passes raw input data to the backend processing pipeline.
3. **Python Engine → Model Storage:** Backend loads pre-trained model binaries (.pkl/.joblib) and feature encoders.
4. **Python Engine Execution:** Backend transforms inputs and calculates customer churn probability.
5. **Python Engine → Streamlit UI:** Backend returns the calculated risk score and churn classification.
6. **Streamlit UI → Analyst:** Frontend updates visual gauge meters and displays retention insights on screen.

3.3 Activity Diagram (System Workflow)

1. **Start:** User launches the Streamlit web interface.
2. **User Action:** User enters customer demographic, billing, and service fields on screen.
3. **Validation Check:** System checks for form completeness.
    - If invalid: System prompts user to fill missing fields.
    - If valid: Data is forwarded to the backend preprocessing pipeline.
4. **Execution:** Preprocessing normalizes features → Trained machine learning model evaluates record.
5. **Output:** System renders risk score meter and displays key churn drivers.
6. **End.**

3.4 State Diagram

- **State 1 (Idle):** Application waiting on the Streamlit dashboard for user inputs.
- **State 2 (Validating):** System validating input parameters and data types upon form submission.
- **State 3 (Predicting):** Running backend inference script and passing inputs into saved model files (.pkl).
- **State 4 (Displaying):** Rendering probability gauge and visual charts before returning to Idle.

3.5 Class Diagram Overview

Class: Customer

- **Attributes***:* customerID, tenure, MonthlyCharges, Contract

Class: ModelEngine

- **Attributes:** model_path, encoder
- **Methods (Functions):** load_model(), predict_churn()

4. Team Responsibilities & Task Allocation

System execution and deliverables are structured across three designated roles:

1. **Member 1 (Mohamed):** Data Preprocessing & ML Models
    - Responsible for dataset cleaning, feature transformation pipelines, training classification models (Logistic Regression, Random Forest, XGBoost), hyperparameter evaluation, and exporting trained model binaries (.pkl/.joblib) for backend handoff.
2. **Member 2 (Catherine):** EDA & BI Dashboards
    - Conducts exploratory data analysis, visual correlation profiling, interactive BI dashboard development (Python, Tableau, Power BI, Looker), and EDA markdown documentation.
3. **Member 3 (Judi – Team Lead):** Database Design, Web App Development & Report Compiler
    - Leads system analysis and design architecture, designs the conceptual ERD diagram, implements the SQL database schema, develops the Streamlit web application, integrates Member 1's ML models into the UI, and compiles the master project report.